Your First Nodes

Take a robot apart from the command line, then build one of your own

Prof. Anis Koubaa

EE 414 — Introduction to Robotics
College of Engineering
Alfaisal University

Fall 2026 — Week 2, Lecture B

How this session works

You type. I type. We stop at every checkpoint until the room is together.

1. A robot you did not write
2. Where your own code lives
3. A publisher
4. A subscriber
5. One command instead of three
6. Breaking it on purpose

A robot you did not write

EE 414 — Introduction to Robotics

Before anything else

```
$ printenv ROS_DISTRO
$ printenv ROS_DOMAIN_ID
$ ros2 topic list
```

Three answers you must see: your distribution name, **your** assigned domain ID, and a list containing `/parameter_events` and `/rosout`.

If `ROS_DISTRO` is empty

You have not sourced the underlay in this terminal. That is not a broken installation — it is a fresh shell. Source it and try again.

Checkpoint 0. Hand up if any of the three is wrong. Do not go on without it.

We start with somebody else's robot

`turtlesim` is a two-dimensional robot simulator that ships with ROS 2 Jazzy. You did not write it, you cannot see its source, and in the next twenty minutes you will find out **exactly** how it is put together.

This is the skill. Not writing nodes — reading a running system.

In the field

In Week 11 you will start Nav2: roughly twenty nodes, none of them yours, and something will not work. The four commands in this section are what you will use. Learning them on a turtle costs you nothing; learning them on Nav2 costs you an afternoon.

Start it

```
ros2 run turtlesim turtlesim_node
```

The ROS 2 command-line tool. Every command this semester begins here.

Start it

```
ros2 run turtlesim turtlesim_node
```

The verb: start one node and keep it in the foreground.

Start it

```
ros2 run turtlesim turtlesim_node
```

The **package** — the installed folder the program was shipped in.

Start it

```
ros2 run turtlesim turtlesim_node
```

The **executable** inside that package. Not the file name — a name the package declared.

A second terminal, and something to drive it

```
$ ros2 run turtlesim turtle_teleop_key
Use arrow keys to move the turtle. 'q' to quit.
```

Two processes. Neither was compiled with the other. Neither names the other anywhere in its code. Press an arrow key and the turtle moves.

What just happened

`turtle_teleop_key` published a `Twist`. `turtlesim_node` subscribed to it. They found each other by **discovery**, because they are on the same domain and agreed on a topic name — and nothing else.

Checkpoint 1. Everyone has a turtle that moves.

Now take it apart — third terminal

```
$ ros2 node list
/turtlesim
/teleop_turtle

$ ros2 topic list
/turtle1/cmd_vel
/turtle1/color_sensor
/turtle1/pose
/parameter_events
/rosout
```

You have just recovered the node list and the topic list of a program you have never seen the source of, from outside, while it runs.

Ask one node what it does

```
$ ros2 node info /turtlesim
Subscribers:
  /turtle1/cmd_vel: geometry_msgs/msg/Twist
Publishers:
  /turtle1/pose: turtlesim/msg/Pose
Services:
  /clear
  /spawn
  /turtle1/set_pen
```

There is Lecture A on one screen: it **subscribes** to a stream of commands, it **publishes** a stream of state, and it offers **services** for things that happen once. Topics for the flow, services for the events.

Read the contract

```
$ ros2 interface show geometry_msgs/msg/Twist
Vector3  linear
Vector3  angular
```

The same type you met this morning, printed by the machine rather than by me. Any topic, any package, any robot: this command tells you what the message actually contains.

In the field

`ros2 interface show` is how you read a vendor's driver when its documentation is a PDF from 2019 and the field names have changed. The installed interface definition cannot be out of date — it is what the code was built against.

Watch the messages go past

```
$ ros2 topic echo /turtle1/cmd_vel
linear:
  x: 2.0
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: 0.0
---
```

Leave this running and press the arrow keys. **Forward** changes `linear.x`; **turn** changes `angular.z`. Every other field stays zero, exactly as Lecture A said it must for a wheeled robot.

Checkpoint 2. Everyone has seen a `Twist` change.

Drive it with no program at all

```
$ ros2 topic pub --rate 1 /turtle1/cmd_vel \
      geometry_msgs/msg/Twist \
      "{linear: {x: 2.0}, angular: {z: 1.8}}"
```

Close the teleop window first. The turtle drives in a circle, commanded from a shell.

Why this is not a party trick

A subscriber can be tested **before its publisher exists**. In Week 6 you will write an obstacle-avoidance node and feed it fake scans this way, on a laptop, without a robot and without a simulator.

Checkpoint 3. Everyone's turtle is driving in a circle.

Twenty minutes, no source code

You found out	With
Which programs are running	`ros2 node list`
What data moves between them	`ros2 topic list`
What one program's job is	`ros2 node info`
What a message actually contains	`ros2 interface show`
What the values are, right now	`ros2 topic echo`
... and you drove it yourself	`ros2 topic pub`

You did not read one line of turtlesim's code. You did not need to.

Now build a system of your own — and it will be inspectable in exactly the same way.

Where your own code lives

EE 414 — Introduction to Robotics

Workspace, package, module

```
ee414_ws/
  src/                    <- YOUR code. Only this is yours.
    ee414_w02_first_node/
  build/                  <- generated. Never edit, never commit.
  install/                <- generated. What actually gets run.
  log/                    <- generated. Build and run logs.
```

A **workspace** is a folder of packages you build together. A **package** is the unit you build, share and install — one folder, its own dependencies, its own nodes. `turtlesim` is a package; yours will sit beside it.

You edit `src/`. `colcon` produces the other three. If you are ever tempted to fix something in `build/`, the answer is in `src/`.

Create the workspace

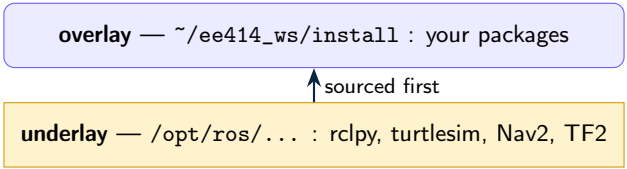
```
$ mkdir -p ~/ee414_ws/src
$ cd ~/ee414_ws
$ colcon build
$ source install/setup.bash
```

Building an empty workspace looks pointless. It is not — it creates `install/` and therefore `install/setup.bash`, which is the file every future terminal will source.

In the field

You will use this workspace for the rest of the semester. Every week adds one package to `src/`. Do not create a new workspace per week — the whole point is that packages accumulate and are used together.

Underlay and overlay



Source the underlay, then your workspace. Your overlay wins where names collide — which is how you can develop a modified version of a system package without uninstalling anything.

In the field

Sourcing the overlay in the wrong order, or sourcing two different workspaces into one shell, produces the strangest bugs in robotics: the code you are editing is not the code that runs. When something is inexplicable, open a clean terminal.

Create the package

```
ros2 pkg create --build-type ament_python
```

Ask about packages. `create` is one verb; `list` and `prefix` are others.

Create the package

```
ros2 pkg create --build-type ament_python
```

A flag, so it needs a value. Leave it out and you get a C++ package by default.

Create the package

```
ros2 pkg create --build-type ament_python
```

The value: a **Python** package.
The alternative, `ament_cmake`, is C++ — not used in this course.

The whole command

```
$ cd ~/ee414_ws/src
$ ros2 pkg create --build-type ament_python \
      --dependencies rclpy std_msgs \
      ee414_w02_first_node
```

The last token is the package name: lower case, underscores, no dashes, because it becomes a Python module name. `--dependencies` lists what the package needs, and writes them into the manifest for you.

In the field

Declaring dependencies now is not bureaucracy. It is how someone else — or you, on the robot in Week 14 — installs what this package needs without guessing.

What it made for you

```
ee414_w02_first_node/
  package.xml                   <- name, version, dependencies
  setup.py                      <- how to install it; ENTRY POINTS live here
  ee414_w02_first_node/         <- your Python module. Code goes HERE.
      __init__.py
  resource/  test/
```

Note the repeated name: the outer folder is the **package**, the inner one is the **Python module**. Your `.py` files go in the inner one.

Under the hood

Putting code in the outer folder is the single most common Week-2 mistake. It builds without complaint and then cannot be found at run time.

A publisher

EE 414 — Introduction to Robotics

talker.py — in ee414_w02_first_node/

```python
import rclpy
from rclpy.node import Node
from std_msgs.msg import String
class Talker(Node):
    def __init__(self):
        super().__init__('talker')
        self.pub = self.create_publisher(String, 'chatter', 10)
        self.create_timer(0.5, self.tick)      # 2 Hz
        self.i = 0
    def tick(self):
        msg = String()
        msg.data = f'hello {self.i}'
        self.pub.publish(msg)
        self.get_logger().info(f'sent: {msg.data}')
        self.i += 1
```

The four lines that make it runnable

```python
def main():
    rclpy.init()
    rclpy.spin(Talker())
    rclpy.shutdown()
```

`rclpy.init()`	join the graph
`rclpy.spin(node)`	hand control to ROS 2: wait for events, call your callbacks
`rclpy.shutdown()`	leave cleanly

Where the loop went

`spin` *is* the loop — it blocks forever, waking to run timer and subscriber callbacks. You never write `while True` in ROS 2: you declare what happens on each event, and `spin` calls you.

Tell the build system the node exists

In `setup.py`:

```
entry_points={
    'console_scripts': [
        'talker = ee414_w02_first_node.talker:main',
    ],
},
```

Read it right to left: run `main`, in module `ee414_w02_first_node.talker`, as a command called `talker`.

In the field

This is where `turtlesim_node` came from. The third token of `ros2 run turtlesim turtlesim_node` is a name somebody declared in a line exactly like the one above. It is also the number-one cause of "executable not found" here: a typo builds cleanly and fails at run time.

Build it and run it

```
$ cd ~/ee414_ws
$ colcon build --symlink-install
$ source install/setup.bash
$ ros2 run ee414_w02_first_node talker
[INFO] [talker]: sent: hello 0
[INFO] [talker]: sent: hello 1
```

Sourcing is not optional and it does not persist. It edits the environment of *one shell*; a new terminal knows nothing. `--symlink-install` links your source instead of copying it, so editing a `.py` file takes effect without rebuilding.

Checkpoint 4. Everyone sees `hello` counting up.

Inspect it the way you inspected the turtle

```
$ ros2 node list
/talker

$ ros2 topic echo /chatter
data: hello 42
---

$ ros2 topic hz /chatter
average rate: 2.001        <- the rate you set, measured
```

Same four commands, your code this time. **You added no logging to make this work** — the graph is inspectable because it is a graph. That property is what will save you in Weeks 9 through 11.

A subscriber

EE 414 — Introduction to Robotics

listener.py

```python
class Listener(Node):
    def __init__(self):
        super().__init__('listener')
        self.create_subscription(String, 'chatter', self.on_msg, 10)

    def on_msg(self, msg):
        self.get_logger().info(f'got: {msg.data}')
```

Four things must match the publisher: the **type** (`String`), the **topic name** (`chatter`), the **QoS** (here, depth 10 — compatible defaults), and the **domain ID**.

Add a second entry point in `setup.py`, rebuild, and run it in a third terminal.

Two nodes, one graph

```
$ ros2 run ee414_w02_first_node listener
[INFO] [listener]: got: hello 87

$ rqt_graph
```

Checkpoint 5. You have drawn your first computation graph.

What you just proved to yourself

You did this	**Which demonstrates**
Started them in either order	No master, no start-up order
Killed the talker; listener survived	A node dies alone
Ran `topic echo` beside both	A third listener needs no change to the publisher
Neither node names the other	They agree on a topic and a type, nothing else

Four of Lecture A's claims, verified by you in twenty minutes. **Try killing the talker now**, and watch the listener wait patiently rather than crash.

One command instead of three

EE 414 — Introduction to Robotics

Launch files

Three terminals for two nodes. A real robot has fifteen. That does not scale, and you cannot reproduce it reliably at 9 a.m. on demo day.

```python
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([
        Node(package='ee414_w02_first_node', executable='talker'),
        Node(package='ee414_w02_first_node', executable='listener'),
    ])
```

In `launch/chatter_launch.py`, plus one line in `setup.py` so the folder is installed. **Note:** this `Node` is a launch description, not the `rclpy` `Node` you subclassed. Same word, different thing.

Run the whole system

```
$ colcon build --symlink-install
$ source install/setup.bash
$ ros2 launch ee414_w02_first_node chatter_launch.py
[INFO] [talker-1]: sent: hello 0
[INFO] [listener-2]: got: hello 0
```

One command. Both nodes. `Ctrl-C` stops both together, and the prefixes tell you which node each line came from.

In the field

Launch files are also where parameters, remappings and namespaces are set — which is how the same node runs twice, on two robots, with different topic names and no code change. Week 8 needs this; Week 11 is impossible without it.

Breaking it on purpose

EE 414 — Introduction to Robotics

Five failures — cause them, read the symptom, fix them

Do this	Symptom	The lesson
New terminal, do not source	`Package not found`	Sourcing is per shell
Misspell the entry point	`No executable found`	The name is a mapping, not magic
Change the listener's domain ID	Both run. Silence.	Discovery is partitioned
Rename the topic in one node only	Both run. Silence.	Names are the whole contract
Set the listener QoS to `BEST_EFFORT`, publisher `RELIABLE`	Both run. Silence.	QoS must be compatible

Three of the five look identical from the outside: two healthy nodes, no data.

The diagnostic order that separates them

```
$ ros2 node list                          # are both alive?
$ ros2 topic list                         # does the topic exist?
$ ros2 topic info /chatter --verbose      # counts, types, QoS
$ printenv ROS_DOMAIN_ID                  # ... in BOTH terminals
```

Publisher count 1, subscriber 0	The subscriber is on another topic or domain
Both counts 1, no data	QoS mismatch — read the profiles
Topic missing entirely	The publisher is not running, or not sourced

The same order you used on the turtle this morning, and the same order in Week 11 on a robot with forty topics.

Before you leave, and before Week 3

Leaving this room, you should have: a workspace, a package, a talker, a listener, a launch file that starts both, and five failures you have caused and fixed.

Assignment 1 is posted today, due Week 3. It extends this package: a publisher at a rate you choose, a subscriber that computes something from what it receives, and a launch file that starts the pair with parameters.

The habit to build now

Commit your workspace to Git today, and commit at the end of every session. In Week 14 your project mark depends partly on a repository history that shows three people working. That history cannot be created retroactively.

Next week: services, parameters, and custom message types.